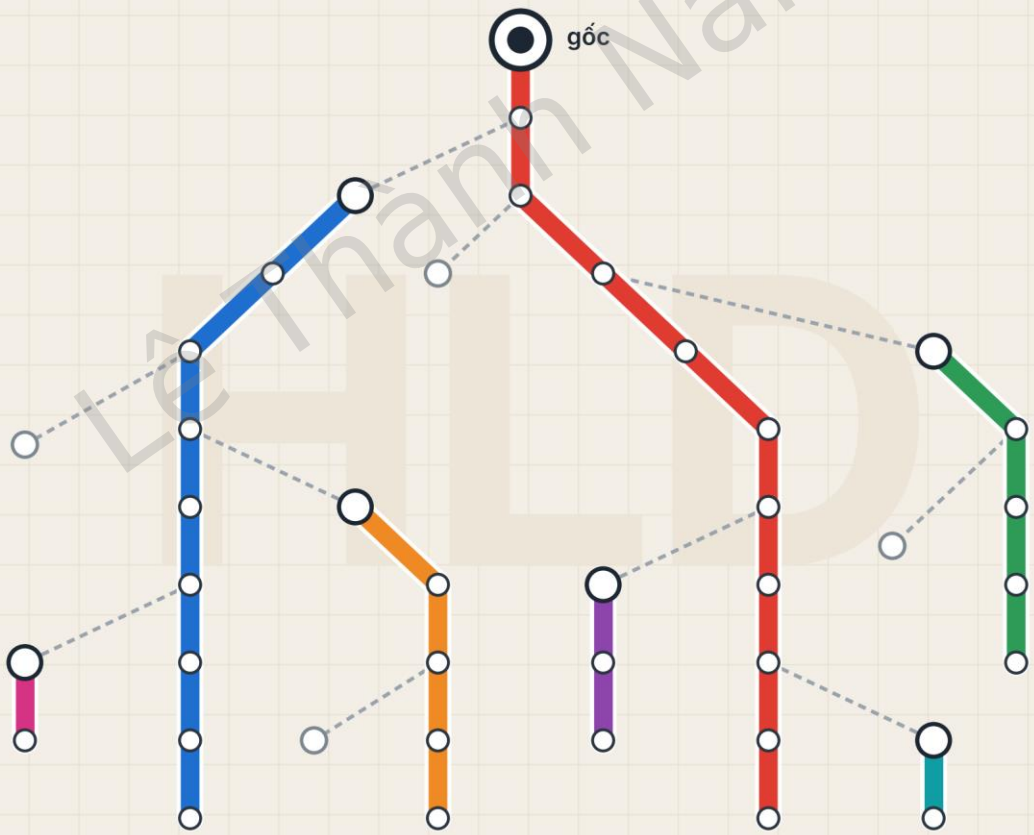
TÀI LIỆU BỒI DƯỠNG HỌC SINH GIỎI TIN HỌC

CHIA NẶNG NHẸ TRÊN CÂY

Heavy-Light Decomposition

Từ cơ bản đến nâng cao

1 **TẬP 1**
LÝ THUYẾT



Lê Thành Nam

chuỗi nặng: một tuyến liên tục

cạnh nhẹ: điểm đổi tuyến

Mọi hành trình về gốc: ≤ $\log_2 n$ lần đổi tuyến

Tổng hợp và biên soạn

Lê Thành Nam

1 Lý thuyết 2 Bài tập 3 Hướng dẫn giải

LỜI NÓI ĐẦU

Trong các đề thi học sinh giỏi Tin học, có một dạng bài xuất hiện ngày càng nhiều: Cho một cây, rồi liên tục thay đổi giá trị ở các đỉnh hoặc các cạnh, xen kẽ với những câu hỏi về đường đi giữa hai đỉnh. "Tổng giá trị trên đường từ u tới v là bao nhiêu?", "Cạnh yếu nhất trên tuyến đường ấy nặng bao nhiêu?", "Hãy cộng thêm x cho mọi đỉnh trên đường đi", "Trên đường đi có bao nhiêu đoạn cùng màu?"... Nếu cây là một đường thẳng, những câu hỏi ấy chỉ là bài toán trên dãy số, giải gọn bằng cây phân đoạn. Nhưng cây thì phân nhánh, và đường đi giữa hai đỉnh có thể rẽ lên rẽ xuống rất khó lường.

Chia nặng nhẹ (Heavy-Light Decomposition, viết tắt HLD) là kĩ thuật biến cái khó ấy thành cái dễ. Nó cắt cây thành những "chuỗi" thẳng, sắp các chuỗi nối đuôi nhau thành một dãy số, và chứng minh được một điều đáng kinh ngạc: Đường đi giữa hai đỉnh bất kì chỉ cắt qua không quá khoảng $2 \cdot \log_2 n$ chuỗi. Nhờ đó, mỗi câu hỏi trên đường đi trở thành một vài câu hỏi trên đoạn của dãy số, và toàn bộ sức mạnh của cây phân đoạn được đem lên cây.

Bộ tài liệu này được viết cho người mới hoàn toàn với HLD. Em chỉ cần biết lập trình C++ cơ bản (mảng, vector, vòng lặp, hàm) và đã từng gặp khái niệm đồ thị. Mọi kiến thức nền khác, từ cây có gốc, duyệt cây, Euler tour cho tới cây phân đoạn, đều được dạy lại từ đầu ở Chương 1. Bộ sách gồm ba tập đi cùng nhau:

- Tập 1: Lý thuyết (cuốn sách này) gồm 12 bài học chia thành 4 chương: Chuẩn bị, đưa cây về dãy số; Chia nặng nhẹ; Ứng dụng của chia nặng nhẹ; Chia nặng nhẹ nâng cao.
- Tập 2: Bài tập gồm 36 bài, mỗi bài học có ba bài tập ở ba mức độ (cơ bản, trung bình, nâng cao), có ràng buộc theo subtask như đề thi thật.
- Tập 3: Hướng dẫn giải phân tích từng bài tập từ cách ngây thơ đến cách tối ưu, kèm chương trình hoàn chỉnh và bảng điểm của từng cách làm.

Mỗi bài học không đưa ra thuật toán ngay. Em sẽ được đặt trước một bài toán cụ thể, thử cách ngây thơ, đếm số phép tính để thấy chỗ bế tắc, rồi được dẫn từng bước để phá bế tắc. Mọi ví dụ đều được chạy tay trên hình vẽ, và cả cuốn sách dùng chung một cây 16 đỉnh để em quen dần với từng đỉnh, từng chuỗi. Mọi chương trình trong sách đều đã được kiểm tra bằng cách đối chiếu với lời giải vét cạn trên hàng trăm bộ dữ liệu ngẫu nhiên, gồm cả những cây "xấu" như đường thẳng dài, hình sao, hình chổi, cây nhị phân đầy đủ, với số hiệu đỉnh bị xáo trộn; các chương trình tối ưu đều đã được chạy trên dữ liệu lớn nhất để bảo đảm thời gian.

Chương trình được viết bằng C++ theo chuẩn C++14, đọc và ghi tệp theo kiểu "có tệp thì đọc tệp" để dùng được cả với phần mềm chấm Themis lẫn các trang chấm trực tuyến. Mọi chương trình duyệt cây đều không dùng đệ quy, để không bị tràn ngăn xếp trên những cây sâu hàng trăm nghìn tầng. Tên biến được đặt ngắn gọn và thống nhất với kí hiệu trong công thức (p, dep, sz, heavy, head, pos, id), để em đọc công thức và đọc chương trình như đọc cùng một thứ.

Chúc các em học tập say mê và đạt kết quả cao!

Lê Thành Nam

Lê Thành Nam

CÁCH HỌC VỚI TÀI LIỆU NÀY

Mỗi bài học đi theo cùng một con đường, giống như cách một người giỏi thật sự suy nghĩ khi gặp bài mới:

Phần	Nội dung
Khởi động	Một câu hỏi nhỏ, gần gũi để em bắt đầu nghĩ.
Bài toán	Đề bài cụ thể, có ví dụ và hình minh họa.
Ý tưởng ngây thơ	Cách làm đầu tiên ai cũng nghĩ ra, kèm phần đếm số phép tính.
Bế tắc	Chỉ ra chính xác vì sao cách ngây thơ chậm, và đặt câu hỏi dẫn tới ý tưởng mới.
Phá bế tắc	Từng bước: Quan sát, công thức, chạy tay, chứng minh, chương trình.
Mở rộng	Các biến thể thường gặp trong đề thi.
Bẫy thường gặp	Những lỗi đã làm mất điểm của nhiều thế hệ học sinh.
Ghi nhớ	Tóm tắt những điều cốt lõi nhất.
Luyện tập	Liên hệ tới các bài tập ở Tập 2 và vài câu tự kiểm tra có đáp số.

Các khung màu trong sách có ý nghĩa cố định:

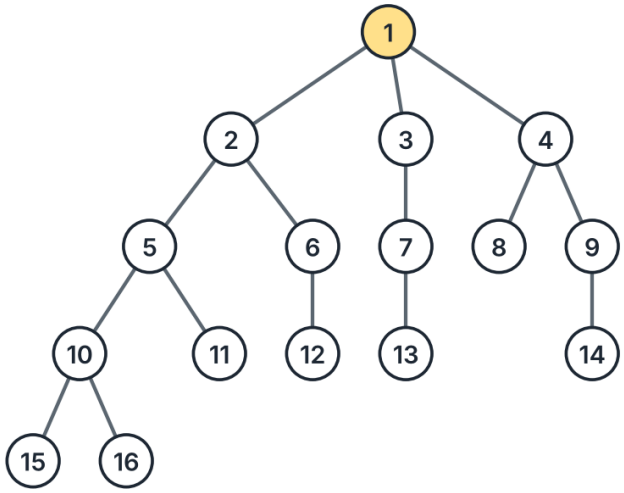
Khung	Ý nghĩa
KHỞI ĐỘNG	Câu hỏi mở đầu bài học.
ĐỊNH NGHĨA, **ĐỊNH LÍ**	Khái niệm mới, phát biểu chính xác.
CÔNG THỨC	Công thức then chốt của bài.
ĐẾM SỐ PHÉP TÍNH	Phân tích độ phức tạp, giải thích vì sao là O(n), O(log n), O($\log^2$ n), ...
BẾ TẮC	Chỗ cách làm hiện tại không vượt qua được.
VÌ SAO ĐÚNG?	Lập luận chứng minh tính đúng đắn.
BẪY THƯỜNG GẶP	Các lỗi hay gặp.
GHI NHỚ	Tóm tắt cuối bài.
ĐỌC THÊM	Nội dung mở rộng dành cho học sinh muốn tìm hiểu sâu hơn.

Cây ví dụ dùng chung. Hầu hết các hình trong sách vẽ trên cùng một cây 16 đỉnh, gốc 1, với mảng cha

p[2..16] = 1 1 1 2 2 3 4 4 5 5 6 7 9 10 10

Hãy chép cây này ra một tờ giấy riêng và để bên cạnh khi học. Từ Bài 4 trở đi, em sẽ tô màu các "chuỗi nặng" của cây này; khi đó tờ giấy ấy sẽ trở thành tấm bản đồ quen thuộc nhất của em.

Lê Thành Nam



Hình F.1. *Cây ví dụ dùng chung: 16 đỉnh, gốc 1.*

Lời khuyên khi học: Hãy đọc với giấy và bút bên cạnh. Mỗi khi gặp một bảng chạy tay, hãy che phần đáp án lại và tự điền trước. Mỗi khi gặp một thuật toán, hãy tự trả lời ba câu hỏi: Chuẩn bị gì trước (tiền xử lý), mỗi câu hỏi làm gì, và vì sao đủ nhanh. Chỉ khi tự viết lại được chương trình mà không nhìn sách, em mới thật sự nắm được bài.

Bản đồ đọc sách. Nếu em đã vững cây phân đoạn và Euler tour, có thể đọc lướt Chương 1 rồi bắt đầu kĩ từ Bài 4. Nếu em mới học lập trình thi đấu, hãy đi tuần tự từng bài: Mỗi bài đều dùng lại những gì của bài trước.

Lê Thành Nam

MỤC LỤC

CHƯƠNG 1. CHUẨN BỊ: ĐƯA CÂY VỀ DÃY SỐ 6
Bài 1. Cây có gốc, kích thước cây con và thứ tự duyệt 7
Bài 2. Duỗi cây: Euler tour và truy vấn trên cây con 18
Bài 3. Cây phân đoạn: Truy vấn và cập nhật trên đoạn 28

CHƯƠNG 2. CHIA NẶNG NHẸ 40
Bài 4. Con nặng, chuỗi nặng và định lí log n 41
Bài 5. Đánh số theo chuỗi: Tổ tiên chung gần nhất và tổ tiên thứ k 51
Bài 6. Truy vấn trên đường đi: Tổng và giá trị lớn nhất 61

CHƯƠNG 3. ỨNG DỤNG CỦA CHIA NẶNG NHẸ 71
Bài 7. Giá trị đặt trên cạnh 72
Bài 8. Cập nhật cả đường đi và cả cây con 80
Bài 9. Khi thứ tự trên đường đi quan trọng 89
Bài 10. Tìm kiếm trên đường đi 99

CHƯƠNG 4. CHIA NẶNG NHẸ NÂNG CAO 108
Bài 11. Khi gốc thay đổi 109
Bài 12. Gộp nhỏ vào lớn: Ý tưởng con nặng ngoài chia chuỗi 117

Lê Thành Nam

CHƯƠNG 1. CHUẨN BỊ: ĐƯA CÂY VỀ DÃY SỐ

Muốn chạy nhanh trên cây, trước hết phải đứng vững trên cây. Chương này chuẩn bị ba thứ mà mọi bài học về chia nặng nhẹ đều cần tới: Biết tính những đại lượng cơ bản của cây (cha, độ sâu, kích thước cây con) mà không cần đệ quy; biết "duỗi" cây thành một dãy số sao cho mỗi cây con trở thành một đoạn liên tiếp; và biết dùng cây phân đoạn để trả lời câu hỏi trên đoạn trong O(log n).

*Ba kiến thức này giống như ba viên gạch móng. Khi đã có chúng, ý tưởng chia nặng nhẹ ở Chương 2 sẽ chỉ còn là một bước nối nhỏ: Thay vì duỗi cây một cách tùy ý, ta duỗi cây một cách thật khéo, sao cho cả **đường đi** cũng trở thành vài đoạn liên tiếp.*

TRONG CHƯƠNG NÀY

Bài 1. Cây có gốc, kích thước cây con và thứ tự duyệt

Bài 2. Duỗi cây: Euler tour và truy vấn trên cây con

Bài 3. Cây phân đoạn: Truy vấn và cập nhật trên đoạn

Lê Thành Nam

BÀI 1 Cây có gốc, kích thước cây con và thứ tự duyệt

Học xong bài này, em sẽ:

✓ Nắm vững các khái niệm cây có gốc, cha, con, tổ tiên, hậu duệ, độ sâu, cây con và kích thước cây con.

✓ Biết lưu cây trong máy bằng danh sách kề, dù đề cho mảng cha hay danh sách cạnh.

✓ Thấy tận mắt vì sao cách "mỗi người báo cáo lên mọi cấp trên" chậm, và đếm được số phép tính của nó.

✓ Tính được độ sâu và kích thước cây con của mọi đỉnh trong O(n) bằng thứ tự duyệt theo chiều rộng.

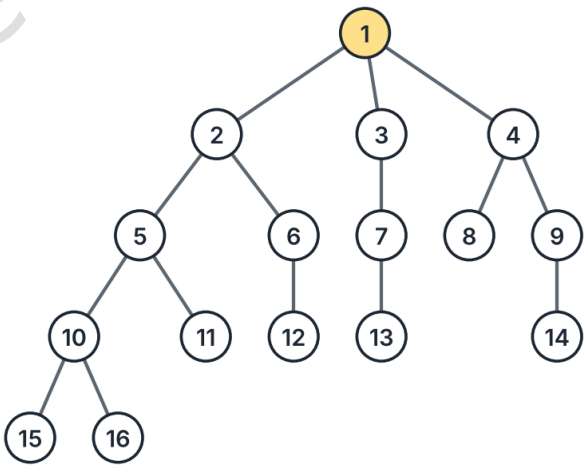
✓ Duyệt được cây theo chiều sâu mà không dùng đệ quy, chuẩn bị cho mọi bài học phía sau.

KHỞI ĐỘNG

Một công ty có 16 nhân viên, sơ đồ tổ chức như Hình 1.1. Người số 1 là giám đốc; mỗi người khác có đúng một cấp trên trực tiếp. Trưởng phòng số 2 tự hào rằng mình "quản lý tám người". Bạn có đồng ý không? Còn trưởng phòng số 4 quản lý bao nhiêu người?

Với sơ đồ 16 người, em đếm bằng mắt trong vài giây. Nhưng nếu công ty có hai trăm nghìn nhân viên, và sơ đồ có thể "dài" tới hàng chục nghìn cấp thì sao? Bài học này sẽ giúp em trả lời câu hỏi ấy cho **mọi** nhân viên chỉ với một lượt duyệt.

Lê Thành Nam

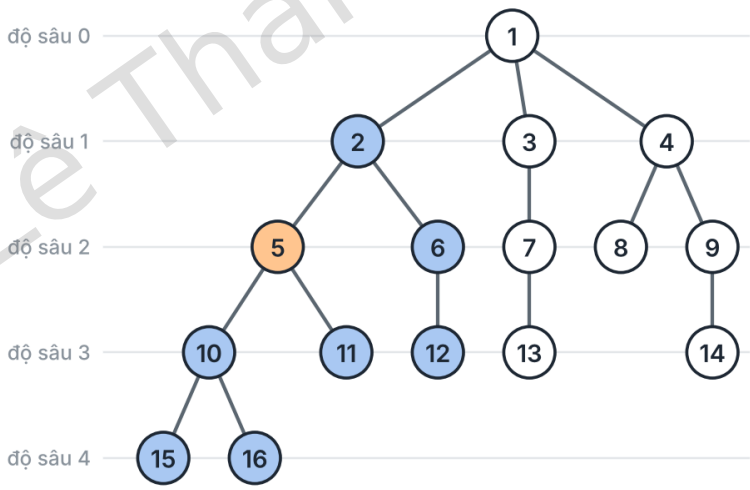


Hình 1.1. *Sơ đồ tổ chức của công ty 16 người. Người số 1 (tô vàng) là giám đốc. Cây này là cây ví dụ dùng chung của cả bộ sách.*

1. Cây có gốc và các thuật ngữ

Một cây là một đồ thị vô hướng, liên thông và không có chu trình. Cây n đỉnh luôn có đúng n − 1 cạnh, và giữa hai đỉnh bất kì có đúng một đường đi đơn (đường đi không lặp lại đỉnh). Khi chọn một đỉnh làm gốc và "treo" cả cây từ gốc xuống, ta được một cây có gốc. Ta dùng các thuật ngữ sau.

Thuật ngữ	Ý nghĩa (ví dụ trên Hình 1.2)
Gốc	Đỉnh được chọn làm đỉnh cao nhất. Ở đây là đỉnh 1.
Cha p[u]	Đỉnh liền trên u trên đường từ u về gốc. p[10] = 5. Gốc không có cha, quy ước p[gốc] = 0.
Con	Các đỉnh nhận u làm cha. Đỉnh 5 có hai con là 10 và 11.
Độ sâu dep[u]	Số cạnh trên đường từ gốc tới u. dep[1] = 0, dep[5] = 2, dep[15] = 4.
Chiều cao h	Độ sâu lớn nhất trong cây. Cây ví dụ có h = 4.
Tổ tiên của u	Các đỉnh nằm trên đường từ u về gốc, **kể cả chính u**. Tổ tiên của 15 là 15, 10, 5, 2, 1.
Hậu duệ của u	Các đỉnh nhận u làm tổ tiên, kể cả chính u.
Lá	Đỉnh không có con: 8, 11, 12, 13, 14, 15, 16.
Cây con gốc u	Tập hợp mọi hậu duệ của u, cùng các cạnh giữa chúng.



Hình 1.2. *Cây gốc 1 với độ sâu ghi bên trái. Cây con gốc 2 được tô xanh: Nó gồm 8 đỉnh 2, 5, 6, 10, 11, 12, 15, 16. Đỉnh 5 (cam) là cha của 10 và 11.*

QUY ƯỚC QUAN TRỌNG

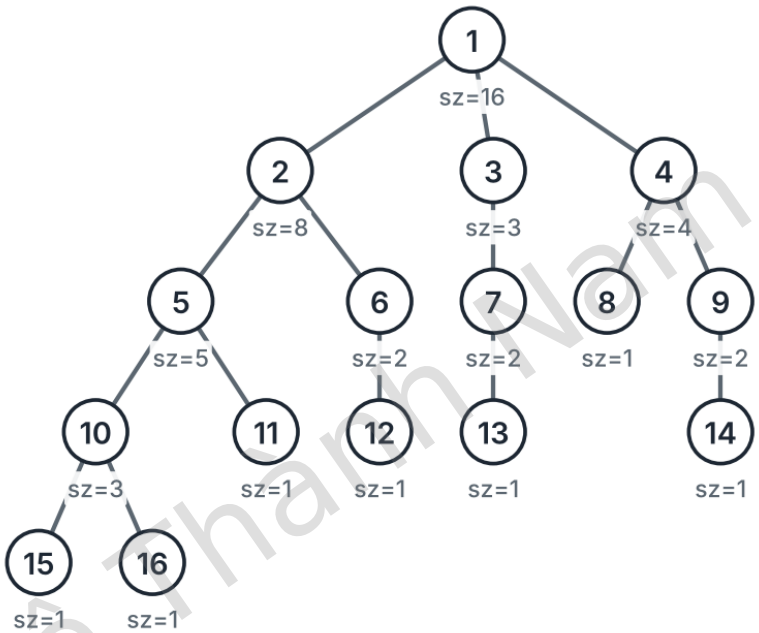
Trong cả bộ sách, mỗi đỉnh được coi là tổ tiên và hậu duệ của chính nó, và cây con gốc u luôn chứa u. Khi cần loại trừ chính u, ta nói "tổ tiên thực sự", "hậu duệ thực sự". Quy ước này làm cho mọi công thức gọn hơn rất nhiều.

Lê Thành Nam

Đại lượng quan trọng nhất của cả bộ sách được định nghĩa ngay dưới đây. Từ Bài 4, em sẽ thấy chính nó quyết định đâu là "đường cao tốc" của cây.

> **ĐỊNH NGHĨA**
> Kích thước cây con của đỉnh u, kí hiệu sz[u], là số đỉnh của cây con gốc u (tính cả u).

Trên Hình 1.2, sz[2] = 8 (đúng như trưởng phòng số 2 nói, nếu tính cả chính anh ấy), sz[4] = 4, sz[1] = 16, và mọi lá có sz bằng 1. Hình Hình 1.3 ghi sz của tất cả các đỉnh.



Hình 1.3. *Kích thước cây con sz[u] ghi dưới mỗi đỉnh. Hãy kiểm tra: sz của mỗi đỉnh bằng 1 cộng tổng sz của các con.*

Lê Thanh Nam

Ba tính chất của sz

Hãy đọc chậm và tự kiểm tra từng tính chất trên Hình Hình 1.3.

1. sz[u] = 1 + tổng sz của các con của u. Cây con gốc u gồm chính u và các cây con của từng con của u; các cây con ấy rời nhau.Ví dụ sz[5] = 1 + sz[10] + sz[11] = 1 + 3 + 1 = 5.

2. sz[gốc] = n và sz[lá] = 1. Đỉnh càng sâu, sz càng nhỏ: Nếu c là con của u thì sz[c] ≤ sz[u] − 1.

3. Tổng sz của mọi đỉnh bằng tổng (dep[v] + 1) của mọi đỉnh. Mỗi đỉnh v được đếm trong sz của đúng dep[v] + 1 tổ tiên của nó. Trên cây ví dụ, cả hai tổng đều bằng 52.

Tính chất 1 là "chìa khóa" của bài học này. Tính chất 3 là "lời cảnh báo": Nếu cây cao, tổng ấy có thể rất lớn, và cách tính nào đếm từng cặp (tổ tiên, hậu duệ) sẽ rất chậm.

2. Lưu cây trong máy

Đề bài thường cho cây theo một trong hai cách:

- **Mảng cha:** n − 1 số p[2], p[3], ..., p[n], gốc là đỉnh 1.
- **Danh sách cạnh:** n − 1 dòng, mỗi dòng gồm hai đỉnh u, v là hai đầu một cạnh. Cạnh không có hướng; ta tự chọn gốc (thường là đỉnh 1).

Trong cả hai trường hợp, ta lưu cây bằng **danh sách kề**: g[u] là danh sách các đỉnh nối với u. Với mảng cha, chỉ cần lưu các con: g[p[v]].push_back(v). Với danh sách cạnh, mỗi cạnh được lưu hai lần, và cha của mỗi đỉnh sẽ được tìm ra khi duyệt từ gốc.

Code ý chính: Hai cách đọc cây

```
vector<vector<int>> g(n + 1);
for (int v = 2; v <= n; v++) {
    cin >> p[v];
    g[p[v]].push_back(v);
}

for (int i = 1; i < n; i++) {
    int u, v;
    cin >> u >> v;
    g[u].push_back(v);
    g[v].push_back(u);
}
```

Đoạn trên dùng khi đề cho mảng cha, đoạn dưới dùng khi đề cho danh sách cạnh. Tổng độ dài các danh sách kề là n − 1 (mảng cha) hoặc 2(n − 1) (danh sách cạnh), nên bộ nhớ chỉ là O(n).

3. Bài toán PHONGBAN

BÀI TOÁN. PHÒNG BAN [PHONGBAN]

Công ty có n nhân viên đánh số từ 1 đến n; người số 1 là giám đốc. Với mỗi v từ 2 đến n, cấp trên trực tiếp của v là p[v]. **Nhóm** của người v gồm chính v và mọi người mà v quản lý trực tiếp hoặc gián tiếp. **Cấp bậc** của người v là số người đứng trên v trong chuỗi quản lý từ v lên giám đốc.

Với mỗi người, hãy cho biết số người trong nhóm và cấp bậc của người đó.

Dữ liệu vào: Dòng đầu gồm số n ($1 \le n \le 2 \cdot 10^5$). Dòng thứ hai gồm n − 1 số p[2], p[3], ..., p[n]. Sơ đồ có thể "dài" tới n cấp.

Kết quả: Gồm n dòng, dòng thứ v ghi hai số sz[v] và dep[v].

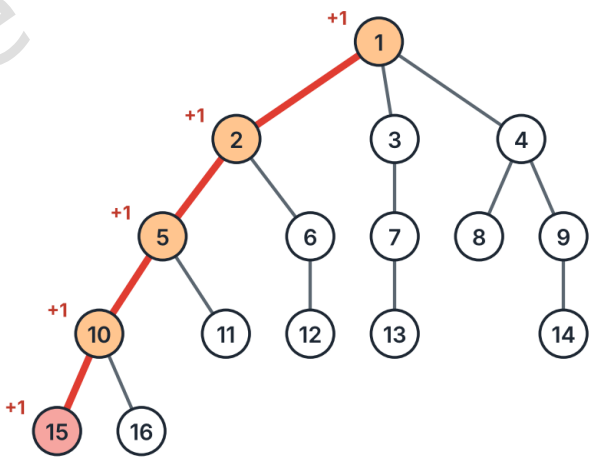
Lê Thanh Nam

Dữ liệu vào	Kết quả	Giải thích
16 1 1 1 2 2 3 4 4 5 5 6 7 9 10 10	16 0 8 1 3 1 4 1 5 2 2 2 2 2 1 2 2 2 3 3 1 3 1 3 1 3 1 3 1 4 1 4	Sơ đồ ở Hình 1.1. Trưởng phòng 2 có nhóm 8 người và cấp bậc 1; trưởng phòng 4 có nhóm 4 người.
5 3 1 5 3	5 0 1 2 4 1 1 3 2 2	Chuỗi quản lý là 4, 5, 3, 1. Chú ý cấp trên có thể mang số hiệu **lớn hơn** cấp dưới.

Ví dụ thứ hai nhắc em một chi tiết dễ bị bỏ qua: **Không có gì đảm bảo p[v] < v.** Vì thế không được duyệt v từ 2 đến n rồi gán dep[v] = dep[p[v]] + 1, vì lúc đó dep[p[v]] có thể chưa được tính.

4. Ý tưởng ngây thơ: Mỗi người báo cáo lên mọi cấp trên

Cách làm đầu tiên ai cũng nghĩ ra là bắt chước định nghĩa: Người v thuộc nhóm của mọi tổ tiên của v. Vậy hãy cho mỗi người v "đi lên" từ v tới giám đốc, và cộng 1 vào sz của từng người đi qua. Số bước đi lên (không tính chính v) cũng chính là cấp bậc của v.



Hình 1.4. *Người 15 đi lên qua 15, 10, 5, 2, 1 và cộng 1 vào sz của cả năm người. Người 16 sẽ đi lại gần như đúng con đường ấy.*

Lê Thanh Nam

Code ý chính: Cách ngây thơ, mỗi người đi lên tới gốc

```
for (int v = 1; v <= n; v++)
    for (int x = v; x != 0; x = p[x]) {
        sz[x]++;
        if (x != v) dep[v]++;
    }
```

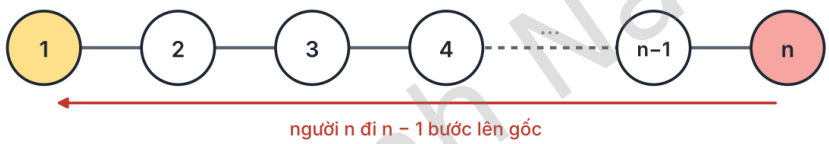
Cách này đúng và không cần biết thứ tự nào cả. Nhưng nó nhanh tới đâu?

ĐẾM SỐ PHÉP TÍNH

Người v đi qua dep[v] + 1 người. Tổng số bước là tổng (dep[v] + 1), đúng bằng tổng sz của mọi đỉnh (tính chất 3).

- Với cây "thấp" (chiều cao h nhỏ), tổng này không quá n·(h + 1): Rất nhanh.
- Với cây đường thẳng 1 – 2 – 3 – ... – n, người thứ v đi v bước, tổng là n(n + 1)/2 ≈ $2 \cdot 10^{10}$ khi n = $2 \cdot 10^5$. Chương trình chạy hàng chục giây.

Cách ngây thơ tốn O(n·h), với h là chiều cao cây; trường hợp xấu nhất là $O(n^2)$.



Hình 1.5. *Cây đường thẳng: Người ở cuối phải báo cáo qua n − 1 cấp. Đây là "kẻ thù" quen thuộc của mọi cách làm leo từng bước trên cây.*

BẾ TẮC

Cả n người cùng đi lên, và họ đi lại cùng những đoạn đường rất nhiều lần: Người 15 và người 16 cùng đi qua 10, 5, 2, 1. Câu hỏi: Có cách nào để mỗi người chỉ báo cáo cho cấp trên trực tiếp, rồi cấp trên gộp số liệu của cả nhóm và báo cáo tiếp một lần duy nhất?

5. Phá bế tắc

Bước 1 Công thức gộp từ dưới lên

Tính chất 1 cho ta đúng điều cần thiết:

CÔNG THỨC

sz[u] = 1 + tổng sz[c] với mọi con c của u

dep[c] = dep[u] + 1 với mọi con c của u, và dep[gốc] = 0.

Hai công thức này đi theo hai chiều ngược nhau. Muốn tính sz[u] thì mọi con của u phải được tính trước u (đi từ dưới lên). Muốn tính dep[c] thì cha của c phải được tính trước c (đi từ trên xuống). Vậy ta cần một thứ tự các đỉnh trong đó cha luôn đứng trước con. Đi xuôi thứ tự ấy để tính dep, đi ngược thứ tự ấy để tính sz.

Lê Thành Nam

Bước 2 Thứ tự duyệt theo chiều rộng (BFS)

Ta lập danh sách order như sau: Bỏ gốc vào order. Lần lượt xét từng đỉnh u trong order (từ đầu tới cuối), bỏ mọi con của u vào cuối order. Danh sách này chính là thứ tự **duyệt theo chiều rộng** (BFS): Gốc, rồi các đỉnh độ sâu 1, rồi độ sâu 2, ...

> **VÌ SAO ĐÚNG?: VÌ SAO TRONG ORDER, CHA LUÔN ĐỨNG TRƯỚC CON?**
>
> Một đỉnh c (khác gốc) chỉ được bỏ vào order vào lúc ta xét cha của nó. Lúc ấy cha của c đã nằm trong order. Vậy cha đứng trước con. Mỗi đỉnh được bỏ vào order đúng một lần (vì nó có đúng một cha), và vì cây liên thông nên mọi đỉnh đều được bỏ vào.

Điều thú vị là danh sách order **vừa là hàng đợi** (ta đọc từ đầu tới cuối) **vừa là kết quả**. Ta không cần cấu trúc queue riêng: Chỉ cần một vector và một chỉ số i chạy từ đầu.

Bước 3 Chạy tay trên ví dụ thứ hai

Với ví dụ thứ hai (p[2..5] = 3 1 5 3), danh sách con là g[1] = {3}, g[3] = {2, 5}, g[5] = {4}. Bảng sau chạy lượt xuôi (tính order và dep).

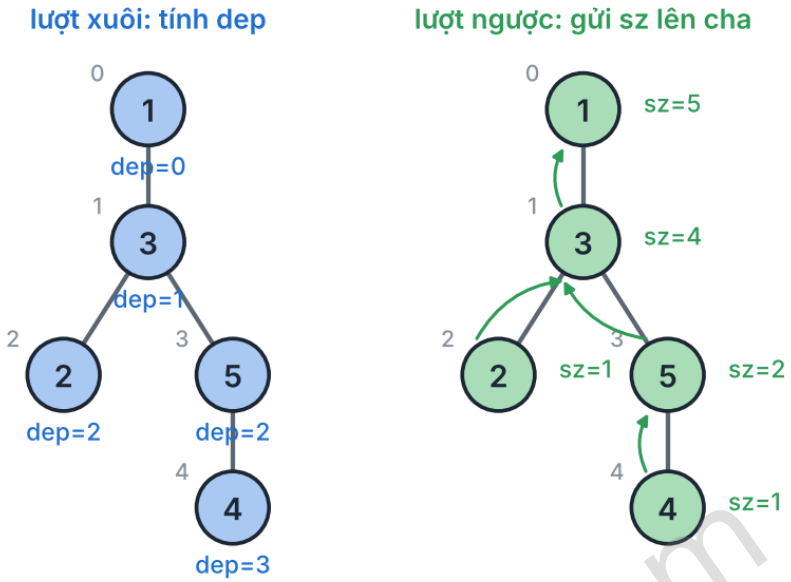
i	u = order[i]	Bỏ thêm vào order	dep vừa tính
0	1	3	dep[3] = 1
1	3	2, 5	dep[2] = 2, dep[5] = 2
2	2	(không có con)	
3	5	4	dep[4] = 3
4	4	(không có con)	

Kết quả order = [1, 3, 2, 5, 4]. Bây giờ đi **ngược** order, mỗi đỉnh v (trừ gốc) gửi sz của mình lên cha. Ban đầu mọi sz bằng 1.

i	v = order[i]	Thao tác
4	4	sz[5] += sz[4]: sz[5] = 1 + 1 = 2
3	5	sz[3] += sz[5]: sz[3] = 1 + 2 = 3
2	2	sz[3] += sz[2]: sz[3] = 3 + 1 = 4
1	3	sz[1] += sz[3]: sz[1] = 1 + 4 = 5

Khi v gửi sz[v] lên cha, sz[v] đã **đầy đủ**: Mọi con của v đứng sau v trong order, nên đã gửi xong trước đó (vì ta đi ngược). Kết quả sz = (5, 1, 4, 1, 2) khớp với đáp án.

Lê Thành Nam



Hình 1.6. *Trái: Lượt xuôi theo order tính dep từ trên xuống. Phải: Lượt ngược gửi sz từ dưới lên. Số nhỏ màu xám là vị trí của đỉnh trong order.*

6. Chương trình hoàn chỉnh

PHONGBAN.CPP: Hai lượt duyệt theo thứ tự BFS, O(n)

```cpp
#include <cstdio>
#include <iostream>
#include <vector>
using namespace std;

int main() {
    ios_base::sync_with_stdio(false);
    cin.tie(nullptr);
    if (fopen("PHONGBAN.INP", "r")) {
        freopen("PHONGBAN.INP", "r", stdin);
        freopen("PHONGBAN.OUT", "w", stdout);
    }
    int n;
    cin >> n;
    vector<int> p(n + 1, 0), sz(n + 1, 1), dep(n + 1, 0);
    vector<vector<int>> g(n + 1);
    for (int v = 2; v <= n; v++) {
        cin >> p[v];
        g[p[v]].push_back(v);
    }
    vector<int> order;
    order.push_back(1);
    for (int i = 0; i < (int)order.size(); i++) {
        int u = order[i];
        for (int v : g[u]) {
            dep[v] = dep[u] + 1;
            order.push_back(v);
        }
    }
    for (int i = n - 1; i > 0; i--) {
        int v = order[i];
        sz[p[v]] += sz[v];
    }
    for (int v = 1; v <= n; v++) cout << sz[v] << ' ' << dep[v] << '\n';
    return 0;
}
```

Võ Thành Nam

ĐẾM SỐ PHÉP TÍNH

Lượt xuôi xét mỗi đỉnh một lần và mỗi cạnh một lần: O(n). Lượt ngược cũng O(n). Với n = $2 \cdot 10^5$, cả chương trình chỉ khoảng 10^6 phép tính và chạy dưới 0,1 giây, dù cây là đường thẳng. So với $2 \cdot 10^{10}$ phép của cách ngây thơ, ta nhanh hơn **hai mươi nghìn lần**.

Hãy để ý cách đọc ở dòng 17 – 20: Ta không lưu p[v] vào g theo chiều ngược lại, vì với mảng cha, g chỉ cần chứa các con. Ở những bài cho danh sách cạnh, ta lưu cả hai chiều và tìm p[v] ngay trong lúc BFS: Khi xét u, mọi đỉnh kề v ≠ p[u] chính là con của u (Bài 2 sẽ dùng cách này).

7. Duyệt theo chiều sâu mà không cần đệ quy

BFS cho ta thứ tự "từng tầng". Nhiều bài học sau (Euler tour ở Bài 2, đánh số chuỗi nặng ở Bài 5) lại cần thứ tự **theo chiều sâu** (DFS): Đi xuống thật sâu theo một nhánh, hết nhánh mới quay lên rẽ sang nhánh khác. Cách viết quen thuộc nhất là hàm đệ quy dfs(u) gọi dfs(c) cho từng con c.

CHÚ Ý: VÌ SAO TRÁNH ĐỆ QUY?

Mỗi lời gọi đệ quy chiếm một phần bộ nhớ ngăn xếp (stack). Trên cây đường thẳng $2 \cdot 10^5$ đỉnh, hàm dfs lồng nhau $2 \cdot 10^5$ tầng. Một số trình chấm (đặc biệt trên Windows, ngăn xếp mặc định khoảng 1 MB) sẽ báo lỗi **tràn ngăn xếp** (stack overflow), và em mất trọn subtask lớn nhất mà không hiểu vì sao.

Ta tự quản lý một ngăn xếp st chứa "đường đi hiện tại" từ gốc tới đỉnh đang đứng, cùng mảng it[u] cho biết u đã đi xuống bao nhiêu con. Ở mỗi bước, nhìn đỉnh u ở đỉnh ngăn xếp: Nếu u còn con chưa đi thì đi xuống con đó; nếu không thì u đã xong, lấy u ra khỏi ngăn xếp (quay lên cha).

Code ý chính: DFS không đệ quy, ghi lại thứ tự đi tới các đỉnh

```
vector<int> st, it(n + 1, 0), pre;
st.push_back(1);
pre.push_back(1);
while (!st.empty()) {
    int u = st.back();
    if (it[u] < (int)g[u].size()) {
        int v = g[u][it[u]++];
        pre.push_back(v);
        st.push_back(v);
    } else {
        st.pop_back();
    }
}
```

Trên cây ví dụ (danh sách con xếp tăng dần), danh sách pre là **1, 2, 5, 10, 15, 16, 11, 6, 12, 3, 7, 13, 4, 8, 9, 14**. Đây gọi là **thứ tự tiền thứ tự** (preorder) của cây. Hãy tự chạy tay vài bước đầu: st = [1] → [1, 2] → [1, 2, 5] → [1, 2, 5, 10] → [1, 2, 5, 10, 15] → 15 là lá, lấy ra → [1, 2, 5, 10] → đi xuống 16 → ...

Lê Thanh Nam

ĐẾM SỐ PHÉP TÍNH

Mỗi đỉnh được bỏ vào và lấy ra khỏi st đúng một lần; mỗi phần tử của danh sách kề được xét đúng một lần qua it[u]++. Tổng cộng O(n), giống BFS, nhưng không có rủi ro tràn ngăn xếp.

Nếu cây cho bằng danh sách cạnh, g[u] chứa cả cha của u. Khi đó chỉ cần thêm một dòng: Nếu v == p[u] thì bỏ qua, ngược lại gán p[v] = u rồi đi xuống. Bài 2 sẽ viết đầy đủ phiên bản này.

8. Mở rộng

Mở rộng 1: Tổng và giá trị lớn nhất trên cây con

Nếu mỗi người có một mức lương a[v], "tổng lương của nhóm v" tính đúng như sz: s[u] = a[u] + tổng s[c]. Giá trị lớn nhất cũng vậy: mx[u] = max(a[u], max mx[c]). Mọi phép toán gộp được (cộng, max, min, xor, ...) đều tính được trên mọi cây con bằng một lượt ngược BFS. Bài tập 1.1 (NHANVIEN).

Mở rộng 2: Cắt một cạnh

Cắt cạnh nối c với cha p[c] chia cây thành hai phần: Cây con gốc c có sz[c] đỉnh, phần còn lại có n − sz[c] đỉnh. Nếu đỉnh có trọng số, tổng hai phần là s[c] và S − s[c], với S là tổng cả cây. Bài tập 1.2 (CATCAY) hỏi cách cắt để hai phần "cân bằng" nhất.

Mở rộng 3: Tổng khoảng cách và kĩ thuật đổi gốc

Gọi f[u] là tổng khoảng cách từ u tới mọi đỉnh khác. Khi "dời" từ cha u xuống con c, có sz[c] đỉnh gần thêm 1 và n − sz[c] đỉnh xa thêm 1, nên **f[c] = f[u] + n − 2·sz[c]**

Tính f[gốc] = tổng dep, rồi đi xuôi order để có f của mọi đỉnh trong O(n). Bài tập 1.3 (TONGKC).

Mở rộng 4: Con có cây con lớn nhất

Với mỗi đỉnh u, hãy tìm con có sz lớn nhất. Trên cây ví dụ: Con lớn nhất của 1 là 2 (sz = 8), của 2 là 5, của 4 là 9. Câu hỏi nhỏ này trông vô hại, nhưng nó chính là hạt giống của cả kĩ thuật chia nặng nhẹ. Hãy ghi nhớ nó; ta sẽ gặp lại ở Bài 4 dưới cái tên **con nặng**.

BẪY THƯỜNG GẶP

Giả sử p[v] < v rồi tính dep[v] = dep[p[v]] + 1 theo thứ tự v tăng dần: Sai ngay ở ví dụ 2.

Quên khởi tạo sz[v] = 1 trước lượt ngược: Mọi kết quả thiếu đúng 1 ở các lá và sai dây chuyền lên trên.

Gửi sz lên cha theo thứ tự xuôi: sz[v] được gửi đi khi chưa nhận đủ từ các con.

Dùng đệ quy để duyệt cây sâu $2 \cdot 10^5$ tầng: Có thể tràn ngăn xếp khi chấm.

Dùng kiểu int cho tổng lương hay tổng khoảng cách: Tổng có thể vượt $2^{31} - 1$, hãy dùng long long.

Dùng endl khi in $2 \cdot 10^5$ dòng: Mỗi endl ép xả bộ đệm, chương trình chậm đi nhiều lần. Hãy dùng '\n'.

Lê Thành Nam

GHI NHỚ

Cây có gốc: mỗi đỉnh (trừ gốc) có đúng một cha; sz[u] là số đỉnh của cây con gốc u (tính cả u).

sz[u] = 1 + tổng sz các con; dep[con] = dep[cha] + 1.

Thứ tự BFS (vector order + chỉ số i) đảm bảo cha đứng trước con: Đi xuôi để tính từ trên xuống, đi ngược để gộp từ dưới lên. Cả hai đều O(n).

Cách "mỗi đỉnh đi lên gốc" tốn O(n·h), có thể tới $O(n^2)$ trên cây đường thẳng.

Duyệt DFS bằng ngăn xếp tự quản lý và mảng it[u] để tránh tràn ngăn xếp.

Luyện tập

Bài	Tên	Mức độ	Gợi ý liên hệ
1.1	NHANVIEN	Cơ bản	Mở rộng 1: Tổng và giá trị lớn nhất trên mọi cây con.
1.2	CATCAY	Trung bình	Mở rộng 2: Cắt một cạnh, so sánh tổng hai phần.
1.3	TONGKC	Nâng cao	Mở rộng 3: Tổng khoảng cách và công thức đổi gốc.

Tự kiểm tra nhanh

1. Trên cây ví dụ, sz[3] và dep[13] bằng bao nhiêu?

(Đáp số: sz[3] = 3, dep[13] = 3.)

2. Tổng sz của mọi đỉnh trên cây đường thẳng 5 đỉnh bằng bao nhiêu?

(Đáp số: 5 + 4 + 3 + 2 + 1 = 15.)

3. Một cây có order (BFS) là [1, 4, 2, 3]. Có thể kết luận 2 là con của 4 không?

(Đáp số: Không. Chỉ biết 4 là con của 1; đỉnh 2 có thể là con của 1 hoặc của 4.)

4. Trên cây ví dụ, f[1] = tổng dep = 36. Hãy tính f[2].

(Đáp số: f[2] = 36 + 16 − 2·8 = 36.)

Lê Thanh Nam